4. AI Halüsinasyonu ve Güven Tuzağı: Kod Denetçisi Olmak

Var Olmayan Kütüphaneleri ve Sahte Metotları Yakalama Sanatı
Bölüm 1 • Zorluk Derecesi: Başlangıç Seviyesi • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir yapay zeka modeli son derece mantıklı görünen ancak gerçekte hiç var olmayan bir npm paketi önerdiğinde ne olur? Bu halüsinasyon bir siber güvenlik saldırısına dönüşebilir mi?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Yapay zeka gerçeği bilmez; olasılıksal olarak en mantıklı gelen bir sonraki kelimeyi tahmin eder. - 📌 Slopsquatting (paket halüsinasyonu), saldırganların sahte kütüphaneler yükleyip şirketleri hacklediği gerçek bir tehdittir. - 📌 Mühendisin en büyük görevi kod yazmak değil, AI çıktısına karşı 'Şüpheci Müfettiş' rolü üstlenmektir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

2023 yılında bir junior geliştirici, Python ile AWS S3 dosyalarını sıkıştırmak için ChatGPT'ye danıştı. Model kendinden çok emin bir dille şu cevabı verdi:

```
pip install aws-s3-fast-compressor
```

Geliştirici kodu projeye ekledi. Ancak bir gariplik vardı: Böyle bir kütüphane Python Package Index'te (PyPI) henüz yoktu! Bir siber güvenlik araştırmacısı bu durumu fark etti ve yapay zekaların sıkça uydurduğu bu sahte kütüphane ismini tescilleyerek içine bir test kodu koydu. Birkaç gün içinde dünyanın önde gelen teknoloji şirketlerinden yüzlerce sunucunun bu sahte paketi indirmeye çalıştığı görüldü!

İşte buna **AI Halüsinasyon Tuzağı** denir. Yapay zeka yalan söylemez; ama var olmayan şeyleri gerçekmiş gibi hayal eder.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Halüsinasyon Tespiti İçin 4 Aşamalı Doğrulama Protokolü:

```
[AI Yanıtı]
   |
   ├── 1. Paket Doğrulaması: NPM / PyPI üzerinde gerçekten var mı? İndirme sayısı
kaç?
   |
   ├── 2. Dokümantasyon Kontrolü: Fonksiyon parametreleri resmi sitede yazıyor mu?
   |
   ├── 3. Tip Kontrolü (Compiler): TypeScript / MyPy hatasız derleniyor mu?
   |
   └── 4. Çalışma Zamanı Testi (Runtime): Unit test uç durumları geçiyor mu?
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Şüpheci Güvenlik ve Kod Denetçisi
Aşağıda bir yapay zekanın önerdiği kod parçası bulunmaktadır.

## GÖREV:
Bu koda asla hemen güvenme. Aşağıdaki analizleri gerçekleştir:
1. Bu kodda kullanılan fonksiyon ve kütüphaneler güncel mi yoksa 'uydurulmuş'
(halüsinasyon) olabilir mi?
2. Hangi satırda sessiz bir mantık hatası (silent bug) veya güvenlik açığı
bulunmaktadır?
3. Bu kodun doğruluğunu kanıtlayacak bağımsız bir test fonksiyonu yaz.

## KOD PARÇASI:
```javascript
const crypto = require('crypto');
function createToken(userId) {
  // AI önerisi:
  return crypto.createCipher('des', userId).final('hex');
}
```
```

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```javascript
// Halüsinasyon / Eski API Tuzağı: createCipher güvensizdir ve Node.js'te deprecated edilmiştir!
const crypto = require('crypto');

function generateSecureToken(userId) {
  // 'des' algoritması kırılmıştır ve createCipher IV (başlatma vektörü) kullanmaz!
  const cipher = crypto.createCipher('des', 'gizli_anahtar');
  let token = cipher.update(userId, 'utf8', 'hex');
  token += cipher.final('hex');
  return token;
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```javascript
// Güncel & Güvenli Standart: AES-256-GCM ve rastgele IV kullanımı
const crypto = require('crypto');

function generateSecureToken(userId, secretKeyBuffer) {
  if (!secretKeyBuffer || secretKeyBuffer.length !== 32) {
    throw new Error("Geçersiz anahtar uzunluğu: 32 baytlık buffer zorunludur.");
  }

  // 12 baytlık benzersiz IV (Initialization Vector)
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', secretKeyBuffer, iv);

  let encrypted = cipher.update(String(userId), 'utf8', 'hex');
  encrypted += cipher.final('hex');
  const authTag = cipher.getAuthTag().toString('hex');

  // IV + AuthTag + Şifreli metin bir arada paketlenir
  return `${iv.toString('hex')}:${authTag}:${encrypted}`;
}
```

💡 **Kod Analizi:** Eski AI modelleri eğitim verilerindeki 10 yıllık eski kodları hatırlayarak güvensiz `crypto.createCipher` metodunu önerebilir. Modern denetim; güvenli `createCipheriv`, AES-256-GCM algoritması ve kimlik doğrulama etiketi (Auth Tag) kullanımını zorunlu kılar.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı
```

- **Tuzak:** AI'ın önerdiği her yeni paketi sorgulamadan package.json'a eklemek.
  - **Çözüm:** npm trends veya Snyk üzerinden paketin yaşını, yapımcısını ve indirilme sayısını kontrol edin.
- **Tuzak:** Derleme hatası vermeyen kodun doğru çalıştığını varsaymak.
  - **Çözüm:** Tip doğrulaması yalnızca şekli kontrol eder; iş mantığını doğrulamak için runtime testi yazın.
- **Tuzak:** AI'a 'Bu kod kesin çalışıyor mu?' diye sorup 'Evet efendim' yanıtına inanmak.
  - **Çözüm:** AI kendi hatasını savunma eğilimindedir; doğrulama için bağımsız test çalıştırın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 4: Yapay Zekayı Yakalamak - Halüsinasyon ve Güvenlik Denetimi*

- 🖥️ **Slayt 1:** Halüsinasyon nedir? Olasılıksal dil modelleri nasıl çalışır?
- 🖥️ **Slayt 2:** Paket halüsinasyonu (Slopsquatting) tehdidi ve tedbirler.
- 🖥️ **Slayt 3:** Eski (deprecated) API'leri tespit etme teknikleri.
- 🖥️ **Slayt 4:** Adım adım kod denetimi: AI çıktısını teste tabi tutmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Modelden bilerek var olmayan bir özellik için kütüphane isteyin. Modelin uydurduğu ismi npm'de aratarak sahte olduğunu gösterin. Ardından aynı işi standart kütüphanelerle güvenli şekilde kodlatın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki `package.json` dosyasını açın. En az 1 kütüphanenin npm sayfasını ziyaret edin; son güncelleme tarihini ve haftalık indirme sayısını inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Paket halüsinasyonu (Slopsquatting) saldırısı nasıl gerçekleşir? - A) Hackerların kullanıcının bilgisayarına fiziksel olarak girmesiyle - B) Saldırganların, yapay zekaların sıkça uydurduğu sahte paket isimlerini tescilleyip içine zararlı kod yerleştirmesiyle - C) Sadece C++ dilinde derleyicinin çökmesiyle - D) Bilgisayar ekranının donmasıyla > **Doğru Yanıt:** B > **Açıklama:** Saldırganlar LLM'lerin uydurmaya meyilli olduğu paket isimlerini takip eder ve bu isimlerle zararlı paketler yayınlayarak dikkatsiz geliştiricilerin indirmesini bekler.

Soru 2: Bir yapay zeka çıktısının güvenilir olduğunu anlamanın en sağlam yolu nedir? - A) AI'a 'Bu doğru mu?' diye sormak - B) Çıktıyı otomatik unit testler, linter'lar ve resmi dokümantasyon ile bağımsızca

doğrulamak - C) Kodun çok uzun ve karmaşık görünmesi - D) Kodun StackOverflow'da bir yanıta benzemesi > **Doğru Yanıt:** B > **Açıklama:** Teknik doğruluk ancak derleyiciler, bağımsız test koşucuları ve resmi dokümantasyon ile garanti altına alınabilir.